



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sesión de Laboratorio 5:

BIT PERMUTATION-PART III



INTRODUCCIÓN A LA CRIPTOGRAFÍA

Profesora || Sandra Díaz Santiago

Grupo || 6CV4

Alumno || Areli Alejandra Guevara Badillo

Octubre 30, 2025

1. CIFRADO CON UNA SPN

EJERCICIO 1

- a) Usa una variable de 32 bits (unsigned int) para almacenar la llave **K**.
La llave debe generarse aleatoriamente y guardarse en un **archivo de texto en formato hexadecimal**.
- b) Una función para **generar aleatoriamente una función S**: $\{0, 1\}^4 \rightarrow \{0, 1\}^4$.
Guarda la tabla generada de S en un archivo de texto, también en formato hexadecimal.

```

1 // Función para generar la llave K
2 unsigned int keyK(FILE *f) {
3     unsigned int K = 0;
4
5     for(int i = 0; i < 32; i++) {
6         int bit = rand() % 2;
7         K = (K << 1) | bit;
8     }
9
10    fprintf(f, "\nK: %08X\n", K);
11    return K;
12 }

```

```

1 // Función para generar la función S
2 void funcionS(FILE *f, unsigned char s[16]) {
3     for(int i = 0; i < 16; i++) {
4         s[i] = i;
5     }
6
7     srand(time(NULL));
8     for(int i = 15; i > 0; i--) {
9         int j = rand() % (i + 1);
10        unsigned char temp = s[i];
11        s[i] = s[j];
12        s[j] = temp;
13    }
14
15    fprintf(f, "z | ");
16    for(int i = 0; i < 16; i++) {
17        fprintf(f, "%X ", i);
18    }
19    fprintf(f, "\n");
20
21    fprintf(f, "S(z) | ");
22    for(int i = 0; i < 16; i++) {
23        fprintf(f, "%X ", s[i]);
24    }
25    fprintf(f, "\n");
26 }

```

Para la función que genera la llave K se realiza generando aleatoriamente bit a bit (valores de 0 y 1) y realizando un corrimiento a la izquierda para colocarlo en el bit menos significativo haciendo uso de un or.

Para la función que genera S() se rellena de 0 a 15 (0 a F en hexadecimal) y después se mezclan utilizando los índices para evitar repetidos.

Los resultados se guardan en un mismo archivo llamado *data.txt*, como se observa a continuación:

```

c cipherSPN.c  data.txt  X
1  z | 0 1 2 3 4 5 6 7 8 9 A B C D E F
2  S(z) | F A 4 8 6 2 C 9 B D 3 0 1 5 7 E
3
4  K: E8C4C5FE

```

EJERCICIO 2

Diseña otro programa para **cifrar** utilizando el algoritmo “Toy encryption”.

- a)** Debe solicitar al usuario el **nombre del archivo** donde está almacenada la llave K y la función S. Le solicitamos al usuario el nombre en el main y con esta función recuperamos los valores de S() y K.

```

1 // Función para leer la función S y la clave K desde un archivo
2 void leerArchivo(FILE *f, unsigned char s[16], unsigned int *k) {
3     char linea[128];
4     unsigned int valor;
5     char *ptr;
6
7     fgets(linea, sizeof(linea), f);
8     fgets(linea, sizeof(linea), f);
9
10    ptr = strchr(linea, '|');
11    ptr++;
12    while (*ptr == ' ') ptr++;
13
14    for (int i = 0; i < 16; i++) {
15        sscanf(ptr, "%x", &valor);
16        s[i] = (unsigned char)valor;
17
18        while (*ptr && *ptr != ' ')
19            ptr++;
20        while (*ptr == ' ')
21            ptr++;
22    }
23
24    fgets(linea, sizeof(linea), f);
25    fgets(linea, sizeof(linea), f);
26    sscanf(linea, "K: %x", k);
27
28    printf("\nValores leídos:");
29    printf("\nz:   ");
30    for (int i = 0; i < 16; i++)
31        printf("%X ", i);
32    printf("\nS(z): ");
33    for (int i = 0; i < 16; i++)
34        printf("%X ", s[i]);
35
36    printf("\n\nK: %08X\n", *k);
37 }

```

- b)** También debe solicitar **M**, que será un carácter ASCII imprimible.

- c)** Como un bloque tiene 8 bits, usa una variable unsigned char para almacenarlo.

```

1 //En main
2 unsigned char M;
3     printf("\nDame un caracter del ASCII imprimible: ");
4     scanf(" %c", &M);

```

Sesión de Laboratorio 5: Bit Permutation-Part III

d) Extrae bloques de 8 bits de la llave K usando **operaciones de bits**.

```
1 //En aplicarToy()
2 unsigned char subk[4];
3 for (int i = 0; i < 4; i++) {
4     subk[i] = (k >> ((3 - i) * 8)) & 0xFF;
5 }
```

Para obtener los valores de cada subllave se realiza un corrimiento a la derecha para que el octeto deseado quede en los 8 bits menos significativos, se le realiza AND con FF y conservamos únicamente con los valores de ese octeto. Este es el orden que se diseño para los valores de cada llave:

$$\begin{array}{cccc} xxxx\ xxxx & xxxx\ xxxx & xxxx\ xxxx & xxxx\ xxxx \\ k_0 & k_1 & k_2 & k_3 \end{array}$$

e) Para realizar la operación de la línea 3, usa la función para sustituir cada bloque de 4 bits en M por los 4 bits correspondientes de la función S almacenada en el archivo de texto.

```
1 // Sustitución usando la función S
2 void sustitucion(unsigned char *m, int tam, unsigned char s[16]) {
3     for(int i = 0; i < tam; i++) {
4         unsigned char b1 = m[i] >> 4;
5         unsigned char b2 = m[i] & 0x0F;
6
7         unsigned char s1 = s[b1];
8         unsigned char s2 = s[b2];
9
10        m[i] = (s1 << 4) | s2;
11    }
12 }
```

f) Imprime C en **hexadecimal**.

```
1 // Función para aplicar el cifrado toy
2 void aplicarToy(unsigned char *M, unsigned int k, unsigned char s[16]) {
3     unsigned char subk[4];
4     for (int i = 0; i < 4; i++) {
5         subk[i] = (k >> ((3 - i) * 8)) & 0xFF;
6     }
7
8     for (int i = 0; i < 3; i++) {
9         *M ^= subk[i];
10        sustitucion(M, 1, s);
11
12        printf("Ronda %d: %02X\n", i + 1, *M);
13    }
14
15    unsigned char C = *M ^ subk[3];
16    printf("Texto cifrado C: %c (%02X)\n", (char)C, C);
17 }
```

Sesión de Laboratorio 5: Bit Permutation-Part III

Ejecución:

```
Nombre del archivo con S() y K: p1/data.txt

Valores leídos:
z:  0 1 2 3 4 5 6 7 8 9 A B C D E F
S(z): 5 8 3 D 2 7 E F 4 B 0 1 A 6 9 C

K: 7BD94366

Dame un caracter del ASCII imprimible: a
Texto claro M: a (61)
Ronda 1: 80
Ronda 2: 7B
Ronda 3: D4
Texto cifrado C:  (B2)
```

Comprobación:

Handwritten notes showing the bit permutation process for the first three rounds of a cipher.

Initial Setup:

z: 0 1 2 3 4 5 6 7 8 9 A B C D E F
S(z): 5 8 3 D 2 7 E F 4 B 0 1 A 6 9 C

Key Schedule:

K: $\begin{matrix} 7 & B & D & 4 \\ 00111011 & 11001001 & 01000011 & 01100110 \end{matrix}$
 k_0 k_1 k_2 k_3

Round 1:

M: a (61) (01100001)
 $k_0 \rightarrow 00111011$
XOR: $\begin{matrix} 00011010 \\ 1010 \end{matrix}$
S(z) → 8 0 (10000000)

Round 2:

M → 10000000
 $k_1 \rightarrow 11001001$
XOR: $\begin{matrix} 01001001 \\ 5 \quad 9 \end{matrix}$
S(z) → 2 B (01111011)

Round 3:

M → 01111011
 $k_2 \rightarrow 01000011$
XOR: $\begin{matrix} 00111000 \\ 3 \quad 8 \end{matrix}$
S(z) → D 4 (11010100)

Final Step:

M → 11010100
 $k_3 \rightarrow 01100110$
C → 10110010 (B2)

XOR Table:

XOR		
0	0	0
0	1	1
1	0	1
1	1	0

2. DESCIFRADO CON UNA SPN

1. Escribe el **pseudocódigo para descifrar** un texto cifrado con el algoritmo de cifrado “Toy”.

Algorithm: Toy Decryption

Input: C a block of ciphertext
 $K = k_0, k_1, k_2, k_3$, where $|k_i| = 8 \text{ bits}$
 S^{-1} the inverse substitution function for S

Output: M the original plaintext block

```

$$M \leftarrow C \oplus k_3$$

$$\text{for } i \leftarrow 2 \text{ down to } 0 \text{ do}$$

$$M \leftarrow S^{-1}(M)$$

$$M \leftarrow M \oplus k_i$$

$$\text{end}$$

$$\text{return } M$$

```

2. Realiza un **ejemplo con una letra M fija**, una llave K y la función S usada en el cifrado, para comprobar que tu pseudocódigo de descifrado es correcto.

Funcionamiento:

$C \leftarrow B \quad 2$

1011 0010

k : 7 B D 9 4 3 6 6

0111 1011 1101 1001 0100 0011 0110 0110

k_0 k_1 k_2 k_3

0 1 2 3 4 5 6 7 8 9 A B C D E F

S^{-1} : A B 4 2 8 0 D 5 1 E C 9 F 3 6 7

$M \leftarrow C \oplus k_3$

$\oplus$ 1 0 1 1 0 0 1 0 C

0 1 1 0 0 1 1 0 k_3

1 1 0 1 0 1 0 0 M

D 4

for

2 M $S^{-1}(M)$ $M \oplus k_i$

2 D 4 3 8 $\oplus$ 0 0 1 1 1 0 0 0 M - 3 8

0 1 0 0 0 0 1 1 $k_2 - 4 3$

0 1 1 1 1 0 1 1 M - 7 B

1 7 B 5 9 $\oplus$ 0 1 0 1 1 0 0 1 M - 5 9

1 1 0 1 1 0 0 1 $k_1 - D 9$

1 0 0 0 0 0 0 0 M - 8 0

0 8 0 1 A $\oplus$ 0 0 0 1 1 0 1 0 M - 1 A

0 1 1 1 1 0 1 1 $k_0 - 7 B$

0 1 1 0 0 0 0 1 M - 6 1

end

$M = 61 (\alpha)$

Se descifro el resultado de la prueba del ejercicio 2 del punto 1.

3. USO DE 3DES-CBC

EJERCICIO 1

Diseña un programa para **generar aleatoriamente una llave para 3DES** y guardarla en un archivo de texto **codificada en base64**.

Usa una función aleatoria de la **biblioteca criptográfica**, no la función rand() del lenguaje.

```
1 # Funcion que genera K (24 bytes - 192 bits)
2 def generarK():
3     K = DES3.adjust_key_parity(get_random_bytes(24))
4
5     with open("p3/K.txt", "wb") as key_file:
6         key_file.write(base64.b64encode(K))
7
8     return
9
10 # Funcion que genera IV
11 def generarIV():
12     IV = get_random_bytes(DES3.block_size)
13
14     with open("p3/IV.txt", "wb") as iv_file:
15         iv_file.write(base64.b64encode(IV))
16
17     return
```

Ejecución:

```
Seleccione una opcion:
1. Generar K y IV
2. Cifrar archivo
3. Descifrar archivo
Opcion: 1
K y IV generados exitosamente.
```

IV.txt	K.txt
1 o9NSDmKnTqM=	1 ZP28B5EyTHaGARWMIDhY6Rpz7NrW30o0

EJERCICIO 2

Desarrolla un programa que **cifre un archivo de texto** usando **3DES-CBC**.

Tu programa debe tomar como entrada el archivo con la **llave**, y el archivo de **texto plano**.

El **texto cifrado** debe guardarse en un archivo de texto **codificado en base64**.

```
1 # Funcion que cifra un archivo usando DES3 en modo CBC
2 def encrypt():
3     print("\nIngrese nombre del archivo a cifrar:")
4     filename = input()
5     with open(filename, "rb") as f:
6         plaintext = f.read()
7
8     print("Ingrese el archivo que contiene la clave K:")
9     key_file = input()
10    with open(key_file, "rb") as kf:
11        key = base64.b64decode(kf.read())
12
13    print("Ingrese el archivo de IV:")
14    iv_file = input()
15    with open(iv_file, "rb") as ivf:
16        iv = base64.b64decode(ivf.read())
17
18    cipher = DES3.new(key, DES3.MODE_CBC, iv)
19    ciphertext = cipher.encrypt(pad(plaintext, DES3.block_size))
20    with open(filename[:-4] + ".enc", "wb") as f:
21        f.write(base64.b64encode(ciphertext))
22
23    return
```

Ejecución:

```
Seleccione una opcion:
1. Generar K y IV
2. Cifrar archivo
3. Descifrar archivo
Opcion: 2

Ingrese nombre del archivo a cifrar:
textos/500KB.txt
Ingrese el archivo que contiene la clave K:
p3/K.txt
Ingrese el archivo de IV:
p3/.txt
IV.txt
Archivo cifrado exitosamente.
```

```
3DEpy 500KB.enc x
1 DEFge4c7M2qxbH2zuRZICZC1Im6SQUQzsuHt7Viwa6grygxcg4MyvW5QrMsZj4aRK5yf
XSoueK5BRN18vknC6o+7QVZTKHu0ukhn5//d+x
+YHzfciAetMMOug4ZADgzH6XICxBlE756tJnAa8pXqR8mn7FXz1D7ozUQYzvmtdtoMLD
xKsV04dJ0tX+A823kdasaEncZikrAvLgTHnEBpLnbbuVUF0RwRn+IJzZ
+0HLFGnhgXS98Vc0puY0PTrm5ENBj+rBrJpZG3F/
X0KzeU3vLXcPqT15uAV5x3PdC1tYicpIEGjA9tqtkFKZuixwUPWepzgrTR8f2lyrbMCUcr
rzbqD10/Gfm2aUSH4F0d98mPlQIAvJj0T6t5AQOT3Hz77FaPcawPNptimxH
+0UpQfTbiyGKM81D+oJDffQ/rXKk/Lwv1Z9Qg2X0v
+jTmVUg6v4B1nrMkJcd42F5mj2UhuXCMmzZb8eQ7u/SqrE+k1NmMseXkK
+6pnMatCvBx9QKFz9h61j03Pym7t2XH1s1ktJyVxhcFYL11MXV/
fRRI4vs9vK5don3mY1VAZfM51nNx358Ka77f03RL56cVer4X1QM65Jza3DZB1s1MBPN
r8YAH21j1zIvW8C3u8+t7peGz71gZ9BF2aa+tBAhm5F/X77Djz1cslUKVXQc/
Thmimo501ehH185pkE48CraA1S4IYBboiD06aIwd8awYQp2CaTwhliQQUbMdk
+3o2soHeCvJmkpaulYcZwKhev1Hu4kgr7FV9VDYdYcV6uVum4L9KHmC1wqufcVxluqr
MQ/+pMnsnR1seu+Cgv/
seraqLoB4xwXmR9cm2e4P3qGkF4YWKCoMULvtQPwY2cqvhSSxPbJvJvmanwOhHyA9y
IHNxcTvuY71X1kRFXp/K63k0gaoQayp5/
DhJ2jL9lmsDxh9KkU853dP3QGTV1UqRC3r003RhGdNZ1/SnUHvesDyExim1mPko8N1
+xpF2rGyTjwZGym8KEgmOodJyeZCr85IMBzF5K4JVFomNOigkjrG6gp/
7qSrdLK9mnpfx3Jv/5ox+6xFab4TWf50GyLcsJv8
+Mr4UhonDyOIkEefH1H3d0Z0vRWC398VC36fXeoJ1a10kjs4xLzBeNPbjvVXU43wpm
Mz1jh00J/
0pzFvnuQj3CJNhtN7xy98MdGkKp4q1xK3cwf1Pi1h1gJyutMREYnLgvbeOPj
+X6x0BYUCwk9c4p1xKJZBUKfc+dnhwzR0dz4blpw0Er/37AB3Q1fsD1ffzpcGZ84
+gmhCG+qPNRy4WSR2HQemayr1krzBzAwU8c8mnTBB1Pfe6yXEBXceGMyXosUDSS/
oZCjZ1GvGSu7mpM030fDvkzrQLMfcsorEF078gYc50yBneJXG6PfdShxjB4QMqdzU
+lvx5Mj8CkbG18k4FDQtk03y6wC1nUksjR1xJHQCdTMj40C9U524P7/
eXeyh0ixb145600IOT1+Ag03DZIC3YFDkgFn8TDd6/KwShAmGPQeJbaaI/eJusRsk
+o0W1vdtHf1gVvJjazeXWzhi2Q2K/zvXy7T
+KqW3C95gV5VcpBDHjJrVxmtp63oGrw75PxOPd3bg1Fr21b20H5EdgxsDy98wXgODEQI
dnMkHQWAPWInby4xo4GuKXey7VnVMB18L6yN0RwJoxNL71rsXaB1vFM2THBJAWMuH
18Sk0GmzPuxqYez2c35C4TimNSoP0IHs1g8q+bfk66p15XBCrvPQg2Z5X
+g4K0XL3fcZXq1tdMFJAVosZ1QLXz3GzD7+7iUu/
R3LuZm3d3T3SjRrRwR1BPNGyud20b21d4fVXYuS6TAAaG8R2821NyMrHuaUrnKa0IYBm
vhdF6BRHwMydh1Qu60nF14DrGXhh0kpFNqutnZonR3JUCYZQW2jJk8vnbAwXED15cL
5yXJX6f0Kof3q6g1EqBkktCFqaiyz9M013gUBU5DWT6R1g70+J8JnKThey/
gi2NPPOxswI9AVpVUobhBBqtEgShbYw2E
vd0Z7eSVTcz0T00bct60Fh1cATMAe80v87MzW/
```

EJERCICIO 3

Desarrolla un programa que **descifre** un texto cifrado usando 3DES.

Tu programa debe tomar como entrada el archivo con la **llave**, y el **texto cifrado**.

El texto plano recuperado debe guardarse en un archivo de texto.

```

1 # Funcion que descifra un archivo usando DES3 en modo CBC
2 def decrypt():
3     print("\nIngrese nombre del archivo a descifrar:")
4     filename = input()
5     with open(filename, "rb") as f:
6         ciphertext = base64.b64decode(f.read())
7
8     print("Ingrese el archivo que contiene la clave K:")
9     key_file = input()
10    with open(key_file, "rb") as kf:
11        key = base64.b64decode(kf.read())
12
13    print("Ingrese el archivo de IV:")
14    iv_file = input()
15    with open(iv_file, "rb") as ivf:
16        iv = base64.b64decode(ivf.read())
17
18    cipher = DES3.new(key, DES3.MODE_CBC, iv)
19    plaintext = unpad(cipher.decrypt(ciphertext), DES3.block_size)
20    with open(filename[:-4] + "_des.txt", "wb") as f:
21        f.write(plaintext)
22
23    return

```

Ejecución:

Seleccione una opcion:

1. Generar K y IV
2. Cifrar archivo
3. Descifrar archivo

Opcion: 3

Ingrese nombre del archivo a descifrar:

textos/500KB.enc

Ingrese el archivo que contiene la clave K:

p3/K.txt

Ingrese el archivo de IV:

p3/IV.txt

Archivo descifrado exitosamente.

```

3DE.py 500KB_des.txt
1  LOS JUEGOS DEL HAMBRE
2  (Saga: "Distritos", vol.1)
3  Suzanne Collins
4  © 2008, The hunger games
5  Traducción: Pilar Ramírez Tello
6  PRIMERA PARTE:
7  LOS TRIBUTOS
8  _____
9  1
10 Cuando me despierto, el otro lado de la cama está frío. Estiro los
11 dedos buscando el calor de Prim, pero no encuentro más que la basta
12 funda de lona del colchón. Seguro que ha tenido pesadillas y se ha met
13 en la cama de nuestra madre; claro que sí, porque es el día de la cose
14 Me apoyo en un codo y me levanto un poco; en el dormitorio entra
15 algo de luz, así que puedo verlas. Mi hermana pequeña, Prim, acurrucad
16 a su lado, protegida por el cuerpo de mi madre, las dos con las mejill
17 pegadas. Mi madre parece más joven cuando duerme; agotada, aunque no
18 tan machacada. La cara de Prim es tan fresca como una gota de agua, ta
19 encantadora como la primula que le da nombre. Mi madre también fue muy
20 guapa hace tiempo, o eso me han dicho.
21 Sentado sobre las rodillas de Prim, para protegerla, está el gato más
22 feo del mundo: hocico aplastado, media oreja arrancada y ojos del color
23 un calabacín podrido. Prim le puso Buttercup porque, según ella, su pe
24 amarillo embarrado tenía el mismo tono de aquella flor, el ranúnculo.
25 gato me odia o, al menos, no confía en mí. Aunque han pasado ya algun
26 años, creo que todavía recuerda que intenté ahogarlo en un cubo cuando
27 Prim lo trajo a casa; era un gatito escuálido, con la tripa hinchada p
28 lombrices y lleno de pulgas. Lo último que yo necesitaba era otra boca
29 alimentar, pero mi hermana me suplicó mucho, e incluso lloró para que
30 dejase quedárselo. Al final la cosa salió bien: mi madre le libró de l
31 parásitos, y ahora es un cazador de ratones nato; a veces, hasta caza
32 alguna rata. Como de vez en cuando le echo las entrañas de las presas,
33 ha dejado de bufarme.

```

3.1 PREGUNTAS

Responde las siguientes preguntas y **añade tus respuestas en el reporte**:

1. ¿Qué biblioteca criptográfica usaste para implementar 3DES-CBC?

Pycryptodome

2. ¿Qué función genera bits pseudorandom en la biblioteca usada?

get_random_bytes()

3. ¿Cuál fue el tamaño de la llave (en bits) que utilizaste?

192 bits

4. ¿Qué mecanismo de relleno (padding) usa tu implementación?

Explica en detalle qué información se añade a un bloque incompleto.

Usa el **mecanismo de relleno PKCS#7** (también conocido como PKCS5Padding cuando el bloque es de 8 bytes, como en DES y 3DES).

Este tipo de padding es el que aplica automáticamente la función:

```
from Crypto.Util.Padding import pad, unpad
```

El relleno PKCS#7 agrega **N bytes**, todos con el mismo valor numérico N, donde N es el número de bytes necesarios para completar el bloque.

Incluso si el texto plano tiene un tamaño múltiplo de 8, el algoritmo **añade un bloque extra completo de padding**.

5. ¿Qué variante de 3DES/TDEA usa la función implementada? (EEE2, EDE2, EEE3, EDE3)

En modo EDE (Encrypt–Decrypt–Encrypt).

6. Al cifrar un texto plano, ¿cuáles son los parámetros que requiere tu función?

Explica el propósito de cada parámetro.

Parámetro	Descripción
key	Clave de 16 o 24 bytes. Se ajusta la paridad mediante <code>DES3.adjust_key_parity()</code> .
mode	Modo de operación, en este caso <code>DES3.MODE_CBC</code> .
iv	Vector de inicialización de 8 bytes obligatorio en el modo CBC.